

- **Ejercicio 1: Ping-Pong (2 procesos).**

- **Descripción:** Limitar la ejecución a 2 procesos. El proceso 0 inicializa una variable, se la envía (MPI_Send) al proceso 1. El proceso 1 la recibe (MPI_Recv), la incrementa y se la devuelve al proceso 0. El proceso 0 la recibe e imprime el valor final.
- **Objetivo:** Uso real de intercambio bidireccional simple entre dos nodos y entendimiento de las parejas Send/Recv y del objeto MPI_Status.

- **Ejercicio 2: Topología de Anillo (El Problema del Deadlock).**

- **Descripción:** Simular un anillo donde hay size procesos. Cada proceso envía un mensaje con su número de rango al proceso (o al 0 si es el último) usando MPI_Send. Inmediatamente después, cada proceso invoca MPI_Recv para recibir el mensaje del proceso.
- **Objetivo:** Provocar un *interbloqueo* intencional. Los sistemas bloquean MPI_Send si el búfer se llena (asignarle un valor muy muy grande para que ocurra el bloqueo), y como todos los procesos intentan enviar antes de que alguien reciba, el programa se quedará "colgado" (*deadlock*).

- **Ejercicio 3: Solución al Deadlock usando MPI_Sendrecv.**

- **Descripción:** Refactorizar el Ejercicio 2 utilizando la función combinada MPI_Sendrecv o MPI_Sendrecv_replace. Los estudiantes deben mapear correctamente quién es el destino (dest) y quién es el origen (source).
- **Objetivo:** Aprender la forma segura recomendada por el estándar MPI para intercambiar datos circulares y prevenir interbloqueos.

- **Ejercicio 4: Despacho de Tareas (Patrón Maestro-Trabajador).**

- **Descripción:** El proceso 0 (maestro) crea un arreglo de elementos y usa un bucle for con MPI_Send para enviar un elemento específico a cada proceso trabajador (del 1 al size - 1). Cada trabajador recibe su parte, la multiplica por 2 y la envía de regreso. El proceso 0 usa MPI_Recv con el comodín MPI_ANY_SOURCE para recolectar las respuestas en el orden que lleguen.
- **Objetivo:** Implementar la lógica fundamental de distribución manual de carga en aplicaciones de alto rendimiento.

- **Ejercicio 5: Reducción Manual Punto a Punto (Entorno Real).**

- **Descripción:** Cada proceso inicializa un valor que simula un "resultado parcial" de un cómputo pesado (ej. el área bajo la curva en un subintervalo para integración, o una suma parcial). Todos los procesos (excepto el 0) envían su resultado parcial al proceso 0 usando MPI_Send. El proceso 0 itera usando MPI_Recv para sumarizar (acumular) las contribuciones de todos en una variable global suma_total.
- **Objetivo:** Mostrar cómo se agrega información global en un sistema de memoria distribuida utilizando únicamente paso de mensajes, sentando las bases empíricas antes de enseñar las comunicaciones colectivas (como MPI_Reduce) en la siguiente etapa del bloque.